

Tópico 1: Python_Atividade1

Disciplina: Projeto e Análise de Algoritmos.
Professor: André Tavares da Silva.
Aluno: Dalton Solano dos Reis.

Enunciado

PAA_TrabalhoPratico1_Ordenacao.pdf

Este notebook reúne a implementação pedida no enunciado em células separadas: leitura do arquivo, BubbleSort com contagem de operações e geração dos casos de teste.

```
In [1]: from __future__ import annotations

import random
from pathlib import Path

def carregar_valores(caminho_arquivo: str | Path) -> list[int]:
    caminho = Path(caminho_arquivo)
    valores: list[int] = []

    with caminho.open("r", encoding="utf-8") as arquivo:
        for numero_linha, linha in enumerate(arquivo, start=1):
            conteudo = linha.strip()

            if conteudo == "":
                break

            try:
                valores.append(int(conteudo))
            except ValueError as exc:
                raise ValueError(
                    f"Valor invalido na linha {numero_linha}: {conteudo!r}"
                ) from exc

    return valores

def bubblesort_operacoes(valores: list[int]) -> int:
    operacoes = 0
    tamanho = len(valores)

    for i in range(tamanho - 1):
        houve_troca = False

        for j in range(tamanho - 1 - i):
            operacoes += 1

            if valores[j] > valores[j + 1]:
                valores[j], valores[j + 1] = valores[j + 1], valores[j]
                houve_troca = True

        if not houve_troca:
            break

    return operacoes

def executar_bubblesort_arquivo(caminho_arquivo: str | Path) -> int:
    valores = carregar_valores(caminho_arquivo)
    operacoes = bubblesort_operacoes(valores)
    print(f"Número de operações: {operacoes}")
    return operacoes
```

```
In [2]: TAMANHOS_PADRAO = (100, 1000, 10000, 100000)

def escrever_valores(caminho: str | Path, valores: list[int]) -> None:
    caminho = Path(caminho)

    with caminho.open("w", encoding="utf-8") as arquivo:
        for valor in valores:
            arquivo.write(f"{valor}\n")
        arquivo.write("\n")

def gerar_arquivos_teste(
    destino: str | Path = "casos_teste",
    tamanhos: tuple[int, ...] = TAMANHOS_PADRAO,
    semente: int = 42,
) -> None:
    random.seed(semente)
    destino_path = Path(destino)
    destino_path.mkdir(parents=True, exist_ok=True)

    for tamanho in tamanhos:
        base = list(range(1, tamanho + 1))
        escrever_valores(destino_path / f"ordenado_{tamanho}.txt", base)
        escrever_valores(destino_path / f"decrecente_{tamanho}.txt", list(reversed(base)))

        randomico = base[:]
        random.shuffle(randomico)
        escrever_valores(destino_path / f"randomico_{tamanho}.txt", randomico)

    print(f"Arquivos gerados em: {destino_path.resolve()}")
```

```
In [3]: # gerar_arquivos_teste()

print(" __ Casos de testes (100, 1.000, 10.000 e 100.000): DECRESCENTE")
executar_bubblesort_arquivo("casos_teste/decrecente_100.txt")
executar_bubblesort_arquivo("casos_teste/decrecente_1000.txt")
executar_bubblesort_arquivo("casos_teste/decrecente_10000.txt")
# executar_bubblesort_arquivo("casos_teste/decrecente_100000.txt")

print(" __ Casos de testes (100, 1.000, 10.000 e 100.000): ORDENADO")
executar_bubblesort_arquivo("casos_teste/ordenado_100.txt")
executar_bubblesort_arquivo("casos_teste/ordenado_1000.txt")
executar_bubblesort_arquivo("casos_teste/ordenado_10000.txt")
# executar_bubblesort_arquivo("casos_teste/ordenado_100000.txt")

print(" __ Casos de testes (100, 1.000, 10.000 e 100.000): RANDÔMICO")
executar_bubblesort_arquivo("casos_teste/randomico_100.txt")
executar_bubblesort_arquivo("casos_teste/randomico_1000.txt")
executar_bubblesort_arquivo("casos_teste/randomico_10000.txt")
# executar_bubblesort_arquivo("casos_teste/randomico_100000.txt")
```

```
__ Casos de testes (100, 1.000, 10.000 e 100.000): DECRESCENTE
Número de operações: 4950
Número de operações: 499500
Número de operações: 49995000
__ Casos de testes (100, 1.000, 10.000 e 100.000): ORDENADO
Número de operações: 99
Número de operações: 999
Número de operações: 9999
__ Casos de testes (100, 1.000, 10.000 e 100.000): RANDÔMICO
Número de operações: 4940
Número de operações: 497355
Número de operações: 49992585
```

Out [3]: 49992585